

ANTHROPIC ACADEMY · 강의노트

MCP 강의노트

Introduction to Model Context Protocol — 한글 정리

Tools · Resources · Prompts / 서버 + 클라이언트 직접 구현

4개 섹션 · 핵심 11개 레슨

1

Introduction

MCP란? · 클라이언트

2

서버 만들기

프로젝트 설정 · 도구 정의 · Inspector

3

클라이언트 연결

클라이언트 구현 · 리소스 · 프롬프트

4

정리 (Wrap up)

3대 프리미티브 리뷰

MCP란? (Introducing MCP)

한 줄 요약 도구를 별도 서버로 분리한 통신 표준 — 도구를 매번 직접 코딩하는 수고를 없앤다.

- 구조: 클라이언트 ↔ 서버 (서버 안에 tools·resources·prompts)
- 도구 정의·실행을 개발자 코드에서 MCP 서버로 분리
- 보통 서비스 제공사가 공식 MCP 서버를 제작·배포
- tool use와 같은 게 아니라 상호보완 (누가 도구를 만드냐에 초점)

cli_project 연결 학습용이라 클라이언트·서버를 둘 다 구현한다 (실무는 보통 한쪽만)

MCP 클라이언트 (MCP Clients)

한 줄 요약 내 서버와 MCP 서버 사이의 중개자. 도구를 직접 실행하지 않고 통신만 잇는다.

- 전송 방식 무관: stdio · HTTP · WebSocket (흔히 stdio)
- 핵심 메시지: list tools(도구 목록) · call tool(실행)
- 흐름: 목록 요청 → Claude에 전달 → 실행 요청 → 결과 회신
- 클라이언트는 실행 주체가 아니라 통신 중개자

cli_project 연결 mcp_client.py 가 이 중개자. chat.py(루프)가 목록·실행을 요청한다

프로젝트 설정 (Project Setup)

한 줄 요약 클라이언트·서버를 한 프로젝트에 같이 만드는 학습용 CLI 챗봇.

- 가짜 문서를 메모리에만 저장 (영속성 없음)
- 서버 도구 2개: 문서 읽기 · 문서 수정
- 준비: zip 해제 → .env에 API 키 → 의존성 설치
- 실행: `uv run main.py` (또는 `python main.py`)

cli_project 연결 지금 받은 그 프로젝트. .env에 ANTHROPIC_API_KEY·CLAUDE_MODEL 필요

도구 정의 (Defining Tools)

한 줄 요약 @mcp.tool 데코레이터로 함수를 도구화. JSON 스키마는 SDK가 자동 생성.

- read_doc_contents: doc_id 받아 내용 반환 (없으면 ValueError)
- edit_document: doc_id·old·new 받아 find/replace
- Field(description=...) 로 인자 설명 (pydantic)
- 패턴: 데코레이터 → 함수 → 타이핑 → 검증 → 로직

cli_project 연결 mcp_server.py 의 TODO 'read a doc / edit a doc' 두 도구

서버 Inspector (The Server Inspector)

한 줄 요약 실제 앱에 붙이지 않고 브라우저에서 서버를 테스트하는 디버거.

- 실행: `mcp dev mcp_server.py` → `localhost:6274`
- Connect → Tools 탭 → 파라미터 입력 → Run Tool
- 별도 스크립트 없이 즉시 반복 테스트
- Claude 연결 전 검증 단계 (UI는 계속 개선 중)

cli_project 연결 서버 TODO 채울 때마다 `mcp dev` 로 검증 → 통과하면 다음

클라이언트 구현 (Implementing a Client)

한 줄 요약 세션(연결)을 감싼 래퍼 클래스. `list_tools.call_tool`은 세션에 위임한다.

- 세션 = SDK가 주는 실제 연결 (단을 때 정리 필요)
- `list_tools` → `session().list_tools()` 의 `.tools`
- `call_tool` → `session().call_tool(name, input)`
- 두 함수는 세션에 일을 떠넘기는 얇은 래퍼

cli_project 연결 `mcp_client.py` TODO. `uv run mcp_client.py` 로 연결·목록 확인

리소스 정의 (Defining Resources)

한 줄 요약 Resources = 읽기용 데이터를 노출하는 기능. HTTP의 GET과 같다.

- Direct(고정 URI): docs://documents — 목록
- Templated({doc_id}): docs://documents/{doc_id} — 단건
- @mcp.resource + mime_type (json / text)
- {param}은 함수 인자로 자동 전달, 반환값 자동 직렬화

cli_project 연결 mcp_server.py 리소스 2개 (≈ GET /todos, GET /todos/{id})

리소스 접근 (Accessing Resources)

한 줄 요약 클라이언트가 리소스를 읽어 와 mime_type 보고 파싱한다.

- read_resource → session().read_resource(AnyUrl(uri))
- contents[0] 꺼내 mime_type 확인
- json이면 json.loads, 아니면 text 그대로
- @멘션한 문서를 프롬프트에 자동 주입 (도구 불필요)

cli_project 연결 mcp_client.py read_resource. cli_chat.py 의 @멘션 처리가 사용

프롬프트 정의 (Defining Prompts)

한 줄 요약 Prompts = 서버가 제공하는 검증된 프롬프트 템플릿. 슬래시 명령으로 호출.

- 서버 제작자가 도메인 맞춤 고품질 프롬프트를 미리 작성
- @mcp.prompt(name, description) + 인자(doc_id)
- 메시지 리스트(user/assistant) 반환 → Claude에 전달
- /format → 문서 선택 → 전용 프롬프트 → Claude가 처리

cli_project 연결 mcp_server.py 프롬프트(format). cli_chat.py 의 /명령어가 호출

클라이언트의 프롬프트 (Prompts in the Client)

한 줄 요약 클라이언트에서 프롬프트 목록 가져오기 구현. 인자를 넘기면 서버가 템플릿에 끼워 넣는다.

- `list_prompts` → `session().list_prompts()` 의 `.prompts`
- `get_prompt(name, args)` → 의 `.messages`
- 인자 → 프롬프트 함수 인자 → 템플릿에 보간
- `/format report.pdf` → 문서 ID가 삽입되어 전달

cli_project 연결 `mcp_client.py` 의 `list_prompts` / `get_prompt` TODO

3대 프리미티브 — 누가 주도하느냐

프리미티브	제어 주체	누구를 위해	용도 / 예시
Tools	모델(Claude)	모델	Claude에 능력 추가 (예: 계산용 코드 실행)
Resources	앱 코드	앱	데이터를 앱으로 (UI·프롬프트 보강, 예: 문서 목록)
Prompts	사용자	사용자	정해진 워크플로 (예: 채팅 시작 버튼·슬래시 명령)

고를 때: Claude 능력 필요 → Tools / 앱 데이터 필요 → Resources / 사용자 워크플로 필요 → Prompts

한 문장 정리

**MCP 서버에 Tools·Resources·Prompts를 정의하고,
클라이언트(세션 래퍼)로 호출해 Claude와 연결한다.**

tool calling 루프는 그대로, 도구의 정의·실행만 별도 서버로 표준화한 것.

다음 강의: *DeepLearning.AI — MCP: Build Rich-Context AI Apps (원격 배포까지)*